[Trung Nguyen] Threads & Synchronization

POSIX Threads on Linux — Reference

---

1. Threads: Concepts, Identification, Creation, Termination

1.1 Definition and memory model

A thread is a mechanism letting one process carry out several tasks concurrently. A process contains at least one thread; a traditional UNIX process is a process with exactly one. All threads in a process run the same program.

Threads share one virtual address space — text, initialized data, BSS, heap. Each thread gets its own stack, but those stacks live inside the same address space, so there is **no hardware barrier**: one thread can read or write another's stack given a pointer. "Private stack" means a private *region*, not a protected one.

1.2 What is shared, what is per-thread

Shared by the whole process	Private to each thread

PID, PPID	Thread ID
PGID, SID	Signal mask
Controlling terminal	Thread-specific data
Credentials (UID/GID)	Alternate signal stack (`sigaltstack()`)
File descriptor table	`errno`
Record locks (`fcntl()`)	Floating-point environment
Signal dispositions	Realtime scheduling policy and priority
`umask`, cwd, root directory	CPU affinity (Linux)
Interval timers, POSIX timers	Capabilities (Linux)
`semadj` values	Stack
Resource limits, nice value	
CPU time (`times()`), resource usage (`getrusage()`)	

The dividing rule: attributes describing the *program's resources or identity* are shared; attributes describing the *state of a running flow of control* are per-thread.

1.3 Threads vs processes

Criterion	Multi-process (`fork()`)	Multi-thread
Data sharing	Needs IPC	Globals / heap
Creation cost	Duplicates page tables and fd table (even with COW)	~10× faster; no page-table duplication
Fault isolation	High	None — one segfault kills the whole process
Synchronization required	Low	Mandatory

Concurrency vs parallelism. Single core: concurrent (interleaved). Multi-core: truly parallel. A thread blocking on I/O does not block the others.

1.4 Why address-space sharing is the root of everything

The kernel gives each process its own virtual address space; the MMU translates virtual to physical addresses through a per-process **page table**. Two processes using the same virtual address 0x601040 land on different physical memory — that is why they cannot collide.

- `fork()` → the kernel builds a **new** page table for the child → two separate address spaces.
- `pthread_create()` → `clone()` with `CLONE_VM` → the new thread **reuses the same page table**. No new table is built.

Three consequences follow directly:

Consequence	Why
Sharing data is just writing a global	Same virtual address → same physical cell
Thread creation is ~10× faster than `fork()`	No new page table — that is the expensive part of `fork()`
One thread segfaults → the whole process dies	No protection boundary between threads

On Linux, threads are implemented via `clone()` with `CLONE_VM`. Two implementations exist: LinuxThreads (old) and NPTL (current). Deviations from the standard: TLPI §33.5.

1.5 Pthreads API conventions — four things to get right

1. Pthreads functions do not set `errno`. They return the error code directly. This is the most common beginner mistake.

2. Compare thread IDs with `pthread_equal()`.

```
if (t1 == t2)                /* WRONG — not portable */
if (pthread_equal(t1, t2))   /* RIGHT */
```

On Linux `pthread_t` is an `unsigned long`, so `==` compiles and works. On other implementations it may be a struct — the code breaks when ported.

3. Never copy Pthreads objects.

```
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER, m2;
m2 = m1;                         /* WRONG — compiles, undefined behaviour */
memcpy(&m2, &m1, sizeof m1);     /* same */
```

`pthread_mutex_t` holds internal state (owner, futex word); copying produces two objects describing one lock state. Practical rule: a struct containing a mutex must not be assigned with `=`, `memcpy`'d, or returned by value — pass it by pointer only.

4. Compile *and* link with `-pthread`, not `-lpthread`. `-pthread` also defines `_REENTRANT` and enables the right code generation.

```
CFLAGS  += -pthread
LDFLAGS += -pthread
```

1.6 Thread identification

- `pthread_self()` returns the calling thread's ID. A thread that wants its own ID must always use this, never read the variable `pthread_create()` wrote into — the two events are unordered (§1.8, problem 2).
- The kernel TID (`gettid()`, the LWP column of `ps -eLf`) is a different namespace from `pthread_t`.
- **IDs are recycled.** Once a thread has terminated and been reaped, its `pthread_t` can be reissued to a new thread.

1.7 Thread creation

```
int pthread_create(pthread_t *thread,            /* [out] receives the ID   */
                   const pthread_attr_t *attr,   /* [in]  NULL = defaults   */
                   void *(*start)(void *),       /* [in]  entry function    */
                   void *arg);                   /* [in]  argument to start */
```

`thread` is an output parameter. The variable must stay alive until you join or detach — do not put it in a function that returns.

`attr` — `NULL` in most cases. Used to change stack size or create detached:

```
pthread_attr_t attr;
pthread_attr_init(&attr);                              /* required */
pthread_attr_setstacksize(&attr, 256 * 1024);          /* instead of the 8 MB default */
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
pthread_create(&tid, &attr, worker, NULL);
pthread_attr_destroy(&attr);                           /* required, avoids a leak */
```

The attr may be destroyed as soon as `pthread_create()` returns — the thread keeps no reference to it. One attr can be reused for many creations.

`start` must have exactly the signature `void *f(void *)`. If it doesn't, fix the signature; do not cast.

`arg` — four ways to pass it

Way	When
A: `NULL`	Nothing to pass

Way	When
B: pointer to a **caller-allocated struct**	**Default choice** for most cases
C: pointer to a **heap-allocated struct**, freed by the thread	The thread outlives the function that created it
D: a small integer cast through `intptr_t`	Only when a single number is genuinely all you need

```
/* B — caller-allocated, lives as long as the threads do */
struct task { int id; char ip[16]; int port; };
struct task tasks[N];
for (int i = 0; i < N; i++) {
    tasks[i].id = i;
    pthread_create(&tids[i], NULL, worker, &tasks[i]);
}

/* C — heap, thread frees it */
struct task *t = malloc(sizeof *t);
pthread_create(&tid, NULL, worker, t);      /* worker: free(t) before return */

/* D — via intptr_t to avoid -Wint-to-pointer-cast on 64-bit */
pthread_create(&tid, NULL, worker, (void *)(intptr_t)i);
```

A string literal is a valid `arg`: it lives in read-only program data for the whole process lifetime, so there is no dangling-pointer question. A local `char buf[32]` is only safe if the creator joins before returning.

1.8 What `pthread_create()` does underneath

1. **Resolve attributes** — from `attr` or defaults (stack size from `RLIMIT_STACK`, typically 8 MB; joinable).
2. **Allocate the stack** — `mmap()` a region plus a **guard page** (no access permissions) at its end, so a stack overflow segfaults instead of silently overwriting a neighbouring region.
3. **Build the thread control structure** — a library-internal struct placed at the top of that same mapping, holding the thread ID, the TLS block, and pointers to `start` and `arg`.
4. **Call `clone()`** with the sharing flags:

Flag	Shares
`CLONE_VM`	Address space (no new page table)
`CLONE_FS`	cwd, root, `umask`

Flag	Shares
`CLONE_FILES`	File descriptor table
`CLONE_SIGHAND`	Signal handlers
`CLONE_THREAD`	Thread group → same PID
`CLONE_SETTLS`	Sets the new thread's TLS register

5. **Kernel creates a `task_struct`**, assigns the kernel TID, queues it in the scheduler.
6. **Return** — write the ID into `*thread` and return 0. The new thread starts in a library wrapper that calls `start(arg)` and calls `pthread_exit()` for you when it returns.

Two things to take away: steps 5 and 6 have **no defined order**, and the stack comes from `mmap()`, so an oversized request or an exhausted address space yields `EAGAIN`.

Error	Cause
`EAGAIN`	Out of resources: `RLIMIT_NPROC`, memory for the stack, or `threads-max`
`EINVAL`	Invalid attr (e.g. stack size below `PTHREAD_STACK_MIN`, usually 16 KB)
`EPERM`	Insufficient privilege for the requested scheduling policy/priority

1.9 Creation pitfalls

Problem 1 — no fault isolation. With `fork()`, a dead child leaves the parent alive. With threads, one crash kills the process.

Problem 2 — the "private" stack is not protected by lifetime. A local variable's address stays *valid-looking* after its function returns, so the read gives garbage rather than a segfault:

```
void spawn(pthread_t *t) {
    int local = 42;
    pthread_create(t, NULL, worker, &local);
}   /* local's lifetime ends here */
```

```
spawn: local = 42, address = 0x7ffcb34693d4
worker: 24637                    ← garbage, no crash
```

Fix: data passed between threads must live in static, global, heap, or caller-allocated storage that outlives the thread.

Problem 3 — the new thread may run before `pthread_create()` returns.

```
void *worker(void *arg) {
    printf("my ID: %lu\n", tid);   /* WRONG — tid may not be written yet */
}
```

Rule: use `pthread_self()`.

Problem 4 — returning from `main()` kills the process. `return` from `main()` is `exit()`, which ends the *process*, not the thread.

Option	Effect	When
`pthread_join(t, NULL); return 0;`	Wait, then exit in a controlled way	Default
`pthread_exit(NULL);` in `main`	`main` ends, other threads keep running, process lives	Rare; loses the ability to return an exit status
`return 0;` immediately	Kills every thread at once	Only when that is the intent

With `pthread_exit()` in `main`, the process ends only when the last thread finishes, and `main`'s stack is **not** reclaimed.

Problem 5 — `PTHREAD_CANCELED` collides with your return value. On glibc it is `((void *) -1)`, so returning `-1` as an error code makes `pthread_join()` report a cancellation that never happened. Fix by putting status in *data*, not in the pointer value:

```
struct result { enum { R_OK, R_FAILED } status; int value; };
void *worker(void *arg) {
    struct result *r = arg;        /* caller-allocated */
    if (failed) { r->status = R_FAILED; return NULL; }
    r->status = R_OK; r->value = 42;
    return NULL;
}
```

This is also why the caller-allocated-struct pattern (way B) is the default: it avoids both the dangling pointer and this collision.

1.10 Thread termination — `pthread_join()`

```
int pthread_join(pthread_t thread, void **retval);
```

Waits for the thread; returns immediately if it has already finished. `retval` receives a copy of the thread's return value, or `NULL` to discard it. Returns 0, or a positive error code.

Three mandatory rules

1. **Joining the same ID twice is undefined** — the ID may already belong to a different thread.
2. **A joinable thread must be joined.** Otherwise it becomes a **zombie thread**, holding resources until creation eventually fails.
3. Only a specific thread can be joined — there is **no "join any"** and **no nonblocking join**.

	`waitpid()` **(process)**	`pthread_join()` **(thread)**
Relationship	Hierarchical — only the parent waits	Peer — any thread may join any thread
Join any	Yes: `waitpid(-1, …)`	**No**
Nonblocking	Yes: `WNOHANG`	**No**
If skipped	Zombie process	Zombie thread

The absence of "join any" is **deliberate**, not an oversight. Threads are not hierarchical, so a "join any" could accidentally join a thread a library created for its own use — the library then loses its thread and later joins an already joined ID. Joining an arbitrary thread is incompatible with modular design.

Why zombies exist. When a joinable thread finishes, the library must retain its return value, its internal thread structure, and its `mmap`'d stack, because someone might still join. Only `pthread_join()` releases them. Each zombie costs one stack (8 MB of address space by default); on 32-bit, a few hundred exhausts the address space and `pthread_create()` returns `EAGAIN`.

Why double-join is worse than it looks.

```
t1: thread A finishes and is joined  → ID 0x7f00 released
t2: thread B is created              → library reissues ID 0x7f00
t3: stale code calls join(0x7f00)    → joins thread B by mistake
```

The call blocks on B (possibly forever, if B is a loop), and B loses its real joiner. This only appears when the program creates and destroys many threads — i.e. under load, never in testing.

1.11 Thread termination — `pthread_detach()`

`pthread_detach()` declares that the thread reclaims itself on exit and its result is not wanted.

```
void *handle_client(void *arg) {
    pthread_detach(pthread_self());
    /* ... */
    return NULL;               /* reclaimed immediately */
```

```
}
```

Why join alone is not enough. In a server, the main thread sits in `accept()` permanently. `pthread_join()` blocks, so joining a worker means not accepting connections; without detaching, every handled client leaves a zombie. Join requires *somebody* to stand and wait, and many architectures have nobody free.

Use `pthread_join()`	Use `pthread_detach()`
The return value is needed	The thread returns nothing
Completion must be observed before continuing	No thread is free to wait
e.g. split an array across 4 threads, join all, combine	e.g. per-client handler, background logger, heartbeat

Process world	Thread world
`wait()` / `waitpid()`	`pthread_join()`
Ignore → zombie process	Ignore → zombie thread
`signal(SIGCHLD, SIG_IGN)`	`pthread_detach()`

Pitfalls

1. Detach is one-way: joining a detached thread is undefined, and so is detaching twice.
2. Detach does not save a thread from `exit()`. It decides **how a thread is reclaimed after it dies, not when it dies**.
3. Do not keep the `pthread_t` of a detached thread — see §1.6.

1.12 `pthread_cancel()` — requesting termination

`pthread_cancel(t)` asks thread `t` to end. By default it uses **deferred cancellation**: the request is recorded, the thread keeps running, and it is only acted on at a **cancellation point** — `pthread_cond_wait()`, `pthread_join()`, `read()`, `write()`, `sleep()`, and similar blocking calls.

This is the correct tool for "end that thread". A signal is not: see §7.1.

---

2. Why Concurrency Is Hard: Race Conditions and Mutual Exclusion

2.1 Three intuitions that are all wrong

Concurrency is hard not because the API is complex, but because three natural assumptions are false:

Intuition	Reality	Resulting problem
"My statement executes as a whole"	`glob++` is three machine instructions and can be cut in half	**Loss of atomicity**
"Once I write, other threads see it"	Store buffers and caches introduce delay	**Memory ordering**
"Instructions run in the order I wrote them"	Compiler and CPU reorder	**Memory ordering**

These three are independent. Solving one does not solve the others.

2.2 Definitions

Critical section — a region of code touching a shared resource whose execution must be atomic: it must not be interleaved with another thread touching the same resource.

Race condition — the program's result depends on the non-deterministic scheduling order between threads.

Data race (C11) — two threads access the same memory location, at least one of them writing, with **no happens-before relationship** → undefined behaviour.

Note: `volatile` does **not** create happens-before. Only mutexes and `_Atomic` do.

2.3 The classic mechanism — read-modify-write

```
Thread 1: loc = glob           (reads 2000)
          ← time slice ends
Thread 2: runs many iterations  (glob reaches 3000)
          ← time slice ends
Thread 1: loc++; glob = loc     (writes 2001) ← all of Thread 2's work is lost
```

`glob++` is not atomic. On RISC (ARM) the compiler still emits load, add, store. Shortening the source does not help.

2.4 Why races are so hard to find

Property	Practical consequence
Non-deterministic	Ten runs, ten results
Hidden at low load	1000 iterations: correct. 10 million: wrong. Light tests find nothing
Architecture-dependent	Correct on x86, broken on ARM (weak memory model)
Optimization-dependent	Works at `-O0`, breaks at `-O2`
No crash	Just a wrong value — no stack trace to follow

Practical conclusion: you cannot rely on testing to find race conditions. Use tooling, and get the design right from the start.

```
gcc -fsanitize=thread -g app.c     # reports the race, not the wrong answer it produced
valgrind --tool=helgrind ./app
```

2.5 What `volatile` does — and does not do

`volatile` only guarantees one thing: every read or write in the source produces a real load/store instruction, never optimized away or merged. Concretely it stops the compiler from:

```
volatile int flag;
while (flag == 0) { }   /* 1. caching it in a register       */
flag = 1; flag = 2;     /* 2. deleting the "redundant" store */
volatile int a, b;
a = 1; b = 2;           /* 3. reordering volatile accesses relative to each other */
```

It does **not** provide atomicity, synchronization, mutual exclusion, or memory ordering between threads. `volatile` is not a concurrency tool.

---

3. Case Study: the Mars Pathfinder Priority Inversion

3.1 The failure

```
t0: LOW-priority task takes the mutex, starts writing to the bus
t1: HIGH-priority task wakes, needs the mutex → blocked, waiting on LOW
t2: MEDIUM-priority task wakes
    → higher priority than LOW → preempts it
    → LOW cannot run → cannot release the mutex
    → HIGH stays blocked
t3: MEDIUM runs long…
    → watchdog sees HIGH miss its deadline
    → concludes the system has failed → RESET
```

The core point: a **medium**-priority task that never touches the mutex ends up blocking a **high**-priority one. The priorities are inverted in practice.

3.2 Why it is worth noting

The mutex behaved exactly as specified. There was no race condition, no deadlock, no logic bug. The problem lives in the **interaction between the mutex and a priority-based scheduler** — a layer the basic mutex API says nothing about.

A mutex guarantees correctness, not timeliness. For a realtime system, correct is not enough.

3.3 The three fixes

Fix	Mechanism	POSIX API
Priority inheritance	While a high-priority task is blocked by a low-priority holder, the holder is temporarily raised to the high priority until it releases	`PTHREAD_PRIO_INHERIT`
Priority ceiling	The mutex carries a priority ceiling; whoever holds it is immediately raised to that level	`PTHREAD_PRIO_PROTECT`
Avoid sharing	Redesign so high- and low-priority tasks do not share a lock	—

---

4. Mutex Locks and Critical Sections

4.1 What a mutex actually is

A synchronization object guaranteeing **mutual exclusion**: at any instant at most one thread is inside the region it protects. It has two states — locked and unlocked — and two atomic operations:

- **lock**: if unlocked, switch to locked and return; if locked, block the caller.
- **unlock**: switch to unlocked and wake one waiter, if any.

4.2 The three guarantees

The most commonly under-appreciated point: a mutex provides three things at once, and all three are needed.

Guarantee 1 — mutual exclusion. One thread in the critical section.

Guarantee 2 — atomicity of a *sequence*. This is what separates a mutex from an atomic variable: it makes a whole block logically indivisible, not one instruction.

```
pthread_mutex_lock(&mtx);
buffer[head] = value;
head = (head + 1) % SIZE;
count++;
pthread_mutex_unlock(&mtx);
```

No cooperating thread can observe an intermediate state — head advanced but count not yet.

Guarantee 3 — memory barrier / happens-before. `pthread_mutex_lock()` contains an acquire barrier, `pthread_mutex_unlock()` a release barrier. Together they establish happens-before between two threads:

```
Thread A                          Thread B
————————                          ————————
data = 42;
unlock(&mtx);  ——release——►  lock(&mtx);  ◄——acquire
                                  read data;   → certain to see 42
```

Without this release–acquire pair, thread B may never see `data = 42`, no matter how many times you write `volatile`.

Consequence: once data is protected by a mutex, you do not need and should not add `volatile`. The mutex already provides everything `volatile` does, and more.

4.3 Ownership

The thread that locks becomes the **owner**; only the owner may unlock. This is not an arbitrary restriction — it has three design consequences:

1. **Clearer code structure.** Lock and unlock sit in one logical flow, easy to check by eye and by tooling.
2. **It enables implementation optimizations.** Because the implementation knows who holds the lock, it can detect self-deadlock, implement recursive mutexes, and support priority inheritance (raising *the holder* requires knowing who that is).
3. **It bounds the applicable problems.** You cannot lock in one thread and unlock in another. If the problem needs that pattern ("thread A grants thread B permission to run"), a mutex is the wrong tool — that is a semaphore's job.

Because of ownership, **acquire/release** is sometimes used instead of lock/unlock.

4.4 Advisory, not mandatory

Neither the kernel nor the hardware enforces a mutex. A thread can ignore it and touch the variable directly. Safety exists only when **every** thread cooperates.

4.5 API

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;      /* static init */
int pthread_mutex_init(pthread_mutex_t *, const pthread_mutexattr_t *);
int pthread_mutex_destroy(pthread_mutex_t *);

int pthread_mutex_lock(pthread_mutex_t *);            /* blocks until acquired */
int pthread_mutex_unlock(pthread_mutex_t *);
int pthread_mutex_trylock(pthread_mutex_t *);         /* EBUSY if held */
```

4.6 When a mutex is the right tool — four conditions, all required

1. **At least two threads touch the same data, and at least one writes.** If every access is a read, no synchronization is needed at all.
2. **The data is several variables that must stay consistent, or the operation is a multi-step read-modify-write.** This is what distinguishes a mutex from an atomic.
3. **No waiting for a condition.** If a thread must wait "until the buffer has data", a condition variable is also needed. A mutex answers "who may enter", never "wait until when".
4. **The critical section is short.** Touch shared data only — no I/O, no heavy computation, no unknown function calls.

4.7 Six programming errors

Error 1 — self-deadlock (re-locking a mutex you hold).

```
pthread_mutex_lock(&mtx);
helper_function();          /* this also locks &mtx → HANG */
pthread_mutex_unlock(&mtx);
```

On Linux the default mutex type **deadlocks silently**; other implementations may return `EDEADLK`. Fixes: do not call other functions while holding a mutex; use `PTHREAD_MUTEX_ERRORCHECK` during debugging to get `EDEADLK` instead of a hang; `PTHREAD_MUTEX_RECURSIVE` is a last resort and usually signals a design problem.

Error 2 — cross-order deadlock.

```
Thread A: lock(m1) → lock(m2)
Thread B: lock(m2) → lock(m1)
```

Strategy	How	Assessment
Lock hierarchy (primary)	Every thread always locks in the **same fixed order**	Efficient; requires discipline
Try-and-back-off	`lock()` the first, `trylock()` the rest; on `EBUSY` release everything, wait, retry	Less efficient; more flexible

If there is no obvious logical order, **impose an arbitrary one** (e.g. by mutex address) and make every thread follow it.

Error 3 — unlocking a mutex you do not own, or that is not locked. Undefined with the default type on Linux; `EPERM` with `ERRORCHECK`.

Error 4 — forgetting to unlock on an early exit. The most common error in real code.

```
/* WRONG */                          /* RIGHT — single exit point */
int process(void) {                 int process(void) {
    pthread_mutex_lock(&mtx);           int ret = 0;
    if (!data) return -1;  /* leak */   pthread_mutex_lock(&mtx);
    if (validate(data) < 0)             if (!data)              { ret = -1; goto out; }
        return -2;         /* leak */   if (validate(data) < 0) { ret = -2; goto out; }
    pthread_mutex_unlock(&mtx);     out:
    return 0;                           pthread_mutex_unlock(&mtx);
}                                       return ret;
                                    }
```

This is one of the few places `goto` in C is correct — the same pattern as resource cleanup in the kernel.

Error 5 — operating on a copy of a mutex. The compiler warns about nothing. See §1.5, note 3.

Error 6 — destroying a mutex still in use.

```
pthread_mutex_destroy(&mtx);   /* while another thread waits on it → UB */
free(obj);                     /* obj contains a mutex → waiters touch freed memory */
```

Rule: destroy only when no thread holds or waits on it; for a mutex in dynamic memory, destroy **before** `free()`.

4.8 Critical-section design

Never call I/O or a blocking function while holding a mutex. This is the root cause of starvation, convoy effect, and priority inversion alike.

```
/* POOR */                                  /* GOOD */
pthread_mutex_lock(&mtx);                   pthread_mutex_lock(&mtx);
result = expensive(shared_data);  /*100ms*/ local = shared_data;
write(fd, buf, len);              /* I/O */ pthread_mutex_unlock(&mtx);
pthread_mutex_unlock(&mtx);                 result = expensive(local);
                                            write(fd, buf, len);
```

Do not call unknown functions while holding a mutex. A user callback may lock another mutex (deadlock), call back into your API (self-deadlock), or do I/O (holds the lock too long).

Granularity

	Too coarse (one mutex for everything)	Too fine (a mutex per variable)
Problem	Parallelism is lost; all threads queue	Many mutexes → high deadlock risk; repeated lock cost
Example	`g_global_lock` for the whole program	One mutex per array element

Rule: **one mutex per group of data sharing an invariant.** No smaller, no larger.

```
/* Right granularity: one mutex per connection — each is an independent unit */
struct connection { pthread_mutex_t lock; int fd; struct buffer buf; int state; };
```

4.9 Fairness: starvation and convoy effect

There is no fairness guarantee. When several threads wait on one mutex, which one acquires it is **unspecified** — no FIFO, no round-robin. If you need fair ordering you must build it yourself (ticket lock, explicit queue).

Problem	Symptom	Cause	Fix
Deadlock	Two threads block each other forever	Inconsistent lock order	Lock hierarchy or try-and-back-off
Starvation	One thread never acquires the mutex	Polling with `trylock`; no fairness guarantee	Avoid `trylock` polling; keep critical sections short
Priority inversion	High-priority task misses its deadline with no deadlock	Scheduler interaction (§3)	`PTHREAD_PRIO_INHERIT`
Convoy effect	Many threads queue behind one slow holder	Long critical section	Shrink the critical section; split the mutex

`trylock` / `timedlock` are rarely used: with a good design the mutex is held briefly and every waiter is served soon, whereas a thread polling with `trylock` risks being starved while the `lock()` queue is served in turn.

---

5. Reader-Writer Locks

> TLPI has no dedicated section on this (only a passing mention in 30.1.6); the material below comes from the POSIX API.

5.1 The problem a mutex handles badly

A device configuration read by 10 threads continuously and written by one thread every few hours:

```
With a mutex:      Thread 1 reads → holds mutex → the other 9 WAIT
With an rwlock:    Threads 1…10 read → all enter at once, genuinely parallel
                   Writer arrives   → waits for every reader, then enters alone
```

Concurrent **reads** cause no race — a race needs a write. A mutex cannot tell the difference, so it blocks readers against each other unnecessarily.

5.2 The concept

An rwlock (shared-exclusive lock) has **three** states, not two:

State	Who may enter
Unlocked	Anyone
Read-locked (shared)	**Many readers at once**; writers wait
Write-locked (exclusive)	Exactly one writer; all readers and other writers wait

Current state	Wants read-lock	Wants write-lock
Free	OK	OK
Read-locked	**OK** ← the whole point	Wait
Write-locked	Wait	Wait

Exactly one cell differs from a mutex — and that cell is the entire value of the mechanism.

5.3 API

```
pthread_rwlock_t rw = PTHREAD_RWLOCK_INITIALIZER;

int pthread_rwlock_init(pthread_rwlock_t *, const pthread_rwlockattr_t *);
int pthread_rwlock_destroy(pthread_rwlock_t *);
```

```
int pthread_rwlock_rdlock(pthread_rwlock_t *);      /* read lock  */
int pthread_rwlock_wrlock(pthread_rwlock_t *);      /* write lock */
int pthread_rwlock_unlock(pthread_rwlock_t *);      /* ONE function for both */

int pthread_rwlock_tryrdlock(pthread_rwlock_t *);   /* EBUSY if unavailable */
int pthread_rwlock_trywrlock(pthread_rwlock_t *);
```

There is only **one unlock function** — the implementation knows which kind you hold.

5.4 When it fits — all of these must hold

1. Shared data with at least one writer.
2. **Read/write ratio heavily skewed** — reads outnumber writes by roughly 10× or more.
3. **The critical section is long enough** for parallelism to matter.
4. **Multiple cores** — on a single core readers do not truly run in parallel, so the benefit is near zero.
5. Readers **genuinely only read** — nothing modified, not even an internal cache.

5.5 It is not always faster than a mutex

A common misconception. The rwlock itself is **more expensive**:

	Mutex	Reader-writer lock
Internal state	One word	More (reader count, writer flag, queues)
Lock/unlock cost	Low	**~2–3× higher**

```
Very short critical section (read one int):
    lock cost >> work inside → rwlock is SLOWER than a mutex, even with many readers

Medium critical section (walk a list, parse a structure):
    work inside is significant → rwlock is clearly faster with many readers
```

Rule: if the critical section reads a couple of simple variables, use a mutex — or `_Atomic`. Use an rwlock only when the reading work genuinely takes time.

5.6 Alternatives

Alternative	Assessment
Mutex	Always correct; only loses reader parallelism. Good enough for most applications
Copy-on-write / double buffer	Readers lock nothing; the writer builds a new copy and atomically swaps the pointer. Fastest, but complex and memory-hungry
RCU (read-copy-update)	Readers cost almost nothing. Very complex, mainly a kernel technique

5.7 Problems in use

Writer starvation — the number one issue. If readers keep arriving, the writer may never acquire the lock:

```
t0: Reader A enters (read-lock)
t1: Writer arrives → waits for A
t2: Reader B arrives → enters anyway (still read-locked)
t3: A finishes
t4: Reader C arrives → enters (B still holds)
…   Writer waits forever
```

POSIX **does not require** writer preference; behaviour is implementation-defined. glibc defaults to PREFER_READER, so the risk is real. On Linux you can change it:

```
pthread_rwlockattr_setkind_np(&attr,
    PTHREAD_RWLOCK_PREFER_WRITER_NONRECURSIVE_NP);
```

(`_np` = non-portable.)

No lock upgrade.

```
/* WRONG — deadlock */
pthread_rwlock_rdlock(&rw);
pthread_rwlock_wrlock(&rw);   /* waits for all readers — including itself */
```

The correct pattern is release, re-acquire, and **re-check** — the same reasoning as `while` with a condition variable: another thread can slip in between:

```
pthread_rwlock_rdlock(&rw);
int need_update = check_condition();
pthread_rwlock_unlock(&rw);
```

```
if (need_update) {
    pthread_rwlock_wrlock(&rw);
    if (check_condition())    /* RE-CHECK — the state may have changed */
        do_update();
    pthread_rwlock_unlock(&rw);
}
```

Readers that quietly write.

```
pthread_rwlock_rdlock(&rw);
cache_hit_count++;            /* A WRITE — many readers run in parallel → race */
value = table.data[i];
pthread_rwlock_unlock(&rw);
```

Any write inside a read-lock is a race. Use `_Atomic` for the counter, or take a write lock.

Recursive read-lock can deadlock. It sounds safe, but under writer preference:

```
Thread A: rdlock #1 → OK
Writer:   wrlock    → queued
Thread A: rdlock #2 → blocked (the writer is queued ahead)
→ A waits for the writer, the writer waits for A → DEADLOCK
```

Rule: do not call unknown functions while holding an rwlock — exactly as with a mutex.

---

6. Reentrancy and Thread-Specific Data

6.1 The root problem: static variables inside library functions

Many traditional C functions keep state in `static` storage:

```
/* simplified asctime() */
char *asctime(const struct tm *tm) {
    static char buf[26];              /* ← SHARED by every call */
    snprintf(buf, sizeof buf, "%s %s %2d …", …);
    return buf;
}
```

```
Thread A: asctime(tm1) → buf = "Mon Aug 31…"
          ← hasn't used the result yet
Thread B: asctime(tm2) → buf = "Tue Sep 01…"   OVERWRITTEN
Thread A: reads the result → gets B's time
```

This is the same class of problem as `errno`: one global carrying state, shared across threads.

6.2 `strtok()` — the canonical example

`strtok()` must remember where it stopped between calls, so that `strtok(NULL, delim)` can continue. It keeps that position in `static` storage.

```
        ┌──────────────────────────────────┐
        │ static char *saveptr;            │  ← ONE shared position
        └──────────────────────────────────┘
             ▲                   ▲
        Thread 1            Thread 2
        strtok("A-B-C")     strtok("1-2-3")   ← trampling each other's state
```

The key insight: `s1` and `s2` are **not** shared — each thread has its own string. The race happens anyway because **`strtok()`'s internal state is shared**. Thread 1's next call can return `"2"` instead of `"B"`.

`strtok_r()` fixes it by moving that state into a variable the **caller** owns:

```
char s[] = "A-B-C";
char *saveptr;                          /* caller-owned, per-thread */
for (char *p = strtok_r(s, "-", &saveptr); p; p = strtok_r(NULL, "-", &saveptr))
    printf("%s\n", p);
```

Generalize: **any function using mutable static storage to carry state between calls risks not being thread-safe** — including any function returning a pointer to static storage.

6.3 Three concepts that are not synonyms

Thread-safe — the function can be called concurrently from several threads and still give correct results. Achieved by any means, **including internal locking**.

```
/* thread-safe via an internal mutex — but NOT reentrant */
static int counter = 0;
static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
int increment(void) {
    pthread_mutex_lock(&mtx);
    int v = ++counter;
    pthread_mutex_unlock(&mtx);
    return v;
}
```

Reentrant — the function can be interrupted midway, called again from the start, and both calls give correct results — **without any locking**. It uses no shared state at all: every datum is in parameters, locals, or caller-supplied memory.

```
int add(int a, int b) { return a + b; }
char *my_strcpy(char *dst, const char *src) {
    char *p = dst;
    while ((*p++ = *src++)) ;
    return dst;
}
```

Reentrant ⟹ thread-safe. The converse is false.

A single rule covers what a reentrant function must avoid: **do not rely on shared mutable state.** Watch for: static mutable data; global mutable data; a shared heap object modified by several execution contexts; state inside a library (`strtok()`); and interfaces returning a pointer to static storage.

Async-signal-safe — the function is safe to call from inside a signal handler. This is the **strictest** requirement, because a handler can interrupt a thread at any point, including while it holds a lock:

```
Thread running:
    pthread_mutex_lock(&malloc_lock)   ← inside malloc()
    ← SIGNAL arrives, handler runs
    handler calls printf() → printf calls malloc()
    → malloc tries to lock &malloc_lock → THIS SAME THREAD holds it
```

```
    → DEADLOCK
```

So **a function that takes a lock is not async-signal-safe**, even if it is thread-safe.

6.4 Unsafe functions and their replacements

Not safe	Use instead
`strtok`	`strtok_r`
`strerror`	`strerror_r`
`localtime`, `gmtime`, `ctime`	`localtime_r`, `gmtime_r`, `ctime_r`
`getpwnam`, `getgrnam`	`getpwnam_r`, `getgrnam_r`
`inet_ntoa`	`inet_ntop`
`gethostbyname`	`getaddrinfo`

6.4 Thread-Specific Data (TSD)

6.4.1 What it is

TSD lets a library keep the `static` variable it needs while giving **each thread its own copy of it** — so the function signature never changes. This is what `strtok_r()` avoids by changing the API instead.

```
              key = 7   ← one key, shared process-wide
                 │
    ┌────────────┼────────────┐
Thread 1      Thread 2      Thread 3
 value:        value:        value:
 0xA100        0xB200        0xC300     ← three separate allocations
```

`pthread_getspecific()` returns `NULL` for a thread that has never called `pthread_setspecific()`. There is no default value — that `NULL` is the basis of the lazy-init pattern.

6.4.2 The standard pattern

```
static pthread_key_t   ctx_key;
static pthread_once_t  ctx_once = PTHREAD_ONCE_INIT;
```

```
struct thread_ctx { char buf[256]; int seq; };

static void ctx_free(void *p) { free(p); }      /* simple and terminating */

static void ctx_key_create(void) { pthread_key_create(&ctx_key, ctx_free); }

struct thread_ctx *ctx_get(void)
{
    pthread_once(&ctx_once, ctx_key_create);

    struct thread_ctx *c = pthread_getspecific(ctx_key);
    if (c == NULL) {
        c = calloc(1, sizeof *c);
        if (c == NULL) return NULL;
        if (pthread_setspecific(ctx_key, c) != 0) {
            free(c);                            /* setspecific can return ENOMEM */
            return NULL;
        }
    }
    return c;
}
```

`pthread_once()` is not decoration. If two threads both run `pthread_key_create()`, the process ends up with two keys: one thread writes key A, the other reads key B, the data disappears and one key leaks permanently.

6.4.3 Order of operations when a thread ends

```
thread terminates
   ├─ 1. cleanup handlers   (pthread_cleanup_push/pop) — LIFO, defined order
   ├─ 2. TSD destructors    — undefined order, repeated rounds
   └─ 3. stack and TCB reclaimed
```

If cleanup must happen in a specific order, use cleanup handlers, not destructors.

6.4.4 The destructor algorithm

It is not "run each destructor once".

```
for (iter = 0; iter < PTHREAD_DESTRUCTOR_ITERATIONS; iter++) {
    any = 0;
    for (each existing key k) {
        value = this thread's value for k;
        if (value != NULL && k has a destructor) {
            set k's value to NULL;        ← FIRST
            call destructor(value);       ← THEN
            any = 1;
        }
    }
    if (!any) break;
}
/* loop exhausted with values still set → silently abandoned */
```

Four consequences.

(a) The value is cleared before the destructor runs. Otherwise a destructor calling `pthread_getspecific()` would see the pointer it just freed. So inside a destructor, `pthread_getspecific(key)` is `NULL` — the data you need is the argument.

```
static void cleanup(void *value)
{
    free(value);        /* use the argument, never getspecific() */
}
```

(b) Multiple rounds, because a destructor may set values again.

```
Round 1: key_A's destructor calls log_error()
         → log_error lazy-inits its own TSD → pthread_setspecific(key_log, buf)
         ⇒ key_log is non-NULL again
Round 2: key_log's destructor frees buf
Round 3: nothing left → stop
```

`PTHREAD_DESTRUCTOR_ITERATIONS` is the cap; POSIX requires **at least 4**, glibc uses exactly 4. A destructor that keeps re-setting its own key is **abandoned after 4 rounds with no warning and no error** — a silent leak.

Rule: a destructor must be simple and terminating. No logging, no calls into unknown APIs that might lazy-init TSD of their own.

(c) Order between keys is undefined. If key_A holds a struct pointing at a buffer owned by key_B, B's destructor may run first and A's then reads freed memory. Either make each key fully

self-contained, or collapse everything into one key:

```
struct thread_ctx {        /* one key, one destructor, order under your control */
    char        *buf;
    FILE        *logfp;
    struct conn *conn;
};
```

(d) A destructor only runs for a non-NULL value. So TSD must hold a **pointer**, never an integer cast to `void *`: storing an fd of `0` (stdin, a valid value) would never be cleaned up.

6.4.5 Four cases where destructors do NOT run

Case	Why
`main()` returns, or any thread calls `exit()`	The process dies; POSIX does not require destructors for the remaining threads, including `main`
`pthread_key_delete(key)`	Only reclaims the key. It does **not** call destructors and does **not** touch any thread's value
Thread killed by a signal whose default action terminates	The process dies
In the child after `fork()`	The other threads do not exist there; their TSD sits in the copied memory with nobody to clean it

The `pthread_key_delete` case deserves a sequence:

```
WRONG:  pthread_key_delete(key);          /* leaks every live thread's data */

RIGHT:  1. ensure every thread using the key has terminated (join them)
        2. their destructors have now run
        3. pthread_key_delete(key);
```

For `main`'s own TSD to be cleaned, end `main` with `pthread_exit(NULL)` instead of `return` (§1.9, problem 4).

6.4.6 Constraints inside a destructor

- Do not call `pthread_exit()` — undefined behaviour.
- The thread is no longer in a normal state: cleanup handlers have already run and part of its context is torn down.
- Destructors still run when a thread is cancelled, but calling a cancellation point inside one gets complicated — avoid blocking calls there.
- **Not async-signal-safe.** A destructor typically calls `free()`, which takes the arena lock. TSD does not make a function safe to call from a signal handler (§6.3).

6.5 TSD vs TLS (`__thread`)

6.5.1 Two different access mechanisms

TLS is known to the compiler and linker. Each `__thread` variable gets a fixed offset in the TLS block, which a hardware register points at (`fs` on x86-64, `TPIDR_EL0` on AArch64).

```
__thread int counter;
counter++;
```

```
mov  %fs:0xfffffffffffffffc, %eax     ; one load
add  $1, %eax
mov  %eax, %fs:0xfffffffffffffffc
```

No function call, no table lookup.

TSD is a library data structure. glibc keeps a two-level array inside `struct pthread`:

```
struct pthread (the thread's TCB)
   ├─ specific_1stblock[32]        ← level 1, preallocated, first 32 keys
   └─ specific[32] ──► block[32]   ← level 2, malloc'd on demand, 1024 keys total
```

Each entry is `{ void *data; uintptr_t seq; }`. `pthread_getspecific()` must split the key into level-1/level-2 indices, check whether the block is allocated, compare `seq` against a global generation counter, and only then return `data`.

`seq` solves a subtle problem: a key can be deleted and later reissued for a different purpose. If a thread still holds a value from the previous incarnation of key 7, the sequence numbers disagree and `NULL` is returned instead of stale data.

6.5.2 Trade-offs

	TSD (`pthread_key_t`)	**TLS (`__thread`)**
Access cost	Function call + two-level lookup + `seq` check	One instruction via a segment register
Automatic destructor	**Yes**	**No** in C
Maximum count	`PTHREAD_KEYS_MAX` — POSIX requires ≥128; Linux 1024	No hard limit
Data type	`void *` only; you allocate the data	A real variable, any type, size fixed at compile time

	TSD (`pthread_key_t`)	TLS (`__thread`)
Initialization	Lazy, written by you	Automatic, from the initialization image
Memory cost	Only for threads that actually use it	**Every thread pays**, even those that never touch the variable
Initial value	Whatever you set	Compile-time constant only (C forbids a dynamic expression)
Requires	libpthread	Compiler + linker + libc TLS support
Safe in a `dlopen`ed library	Always	Constrained — see 6.5.4

6.5.3 How `__thread` initialization works

```
__thread int  counter = 0;
__thread char buf[256];
__thread int  x = compute();      /* compile error in C — must be a constant */
```

The linker builds an **initialization image** of the whole TLS segment (`.tdata` for initialized values, `.tbss` for zeroed ones). Creating a thread `memcpy`s that image into the new TLS block, so every thread starts from an independent copy.

Memory consequence, which matters on embedded targets: `__thread char buf[4096]` costs **every** thread 4 KB, including threads that never touch `buf`. Fifty threads is 200 KB. TSD only allocates for the threads that call in.

6.5.4 TLS models

The compiler emits one of four access sequences, selectable with `-ftls-model=` or `__attribute__((tls_model("…")))`:

Model	Access	Usable when
`local-exec`	Constant offset from the thread pointer — fastest	The variable is in the executable itself
`initial-exec`	One GOT load, then an offset	Library linked at startup, **never** `dlopen`ed
`local-dynamic`	Calls `__tls_get_addr`	Shared library, variable in the same module
`global-dynamic`	Calls `__tls_get_addr` — slowest, default for shared libraries	Any case

The common real failure: a `.so` built with `initial-exec` for speed, later loaded with `dlopen()`. The static TLS block was sized at process startup and may have no room left:

```
dlopen failed: cannot allocate memory in static TLS block
```

TSD never has this class of failure. If you are writing a library that might be `dlopen`ed, that is a concrete reason to choose TSD.

6.5.5 The C11 standard spellings

```
#include <threads.h>

thread_local int counter;              /* = __thread */

tss_t key;                             /* = pthread_key_t */
tss_create(&key, destructor);
tss_set(key, p);
void *p = tss_get(key);
tss_delete(key);
```

C11's `thread_local` still has no destructor. C++ differs: its `thread_local` runs a constructor and destructor per thread, which largely removes the reason for TSD in C++.

Practical caveat: `<threads.h>` support is uneven — glibc only since 2.28, and many embedded toolchains lack it. In embedded C, `__thread` plus `pthread_key_t` remains the safer choice.

6.5.6 Choosing

```
Data needs cleanup when the thread dies (malloc, fd, lock)?  → TSD, with a destructor
Small POD, many threads, access speed matters?               → __thread
Library that might be dlopen'ed?                             → TSD
Need both speed AND cleanup?                                 → combine them
```

The combination is a real technique used in glibc and other runtimes: `__thread` for the fast access path, a TSD key purely to obtain a destructor.

```
static __thread struct ctx *tls_ctx;   /* fast access */
static pthread_key_t        dtor_key;  /* exists only to get a destructor */

struct ctx *ctx_get(void)
{
    if (tls_ctx == NULL) {
        tls_ctx = calloc(1, sizeof *tls_ctx);
        pthread_setspecific(dtor_key, tls_ctx);   /* register for cleanup */
    }
```

```
    return tls_ctx;                  /* every later call: one load */
}
```

The destructor must also set `tls_ctx = NULL`, or later destructor rounds may see a dangling pointer.

7. Threads and Signals, fork, and I/O

Common ground for all three: they arise at the **boundary** between what a process shares and what each thread owns.

```
PROCESS
┌──────────────────────────────────────────────┐
│ SHARED (every thread sees the same)          │
│  • Memory (heap, globals)                    │
│  • File descriptor table                     │
│  • Signal handler (disposition) table        │
│  • cwd, PID, UID                             │
├──────────────────────────────────────────────┤
│  Thread 1     Thread 2     Thread 3          │
│  ┌────────┐   ┌────────┐   ┌────────┐        │
│  | stack  |   | stack  |   | stack  |        | ← PRIVATE
│  | regs   |   | regs   |   | regs   |        │
│  | errno  |   | errno  |   | errno  |        │
│  | mask   |   | mask   |   | mask   |        │
│  └────────┘   └────────┘   └────────┘        │
└──────────────────────────────────────────────┘
```

7.1 Threads and signals

Rule 1 — dispositions are shared, masks are per-thread. There is one `my_handler()` for the whole process; you cannot have Thread 1 react to `SIGINT` one way and Thread 2 another. The only per-thread control you have is **who is eligible to receive it**. A new thread inherits a *copy* of its creator's mask, independent thereafter.

Rule 2 — two routing modes.

Mode A — the signal is generated by the thread itself: `SIGSEGV`, `SIGBUS`, `SIGFPE`, `SIGILL`, `SIGPIPE`, plus `pthread_kill(tid, sig)` where you name the target. Delivered to the culprit.

Mode B — the signal comes from outside (`kill(pid, …)`, Ctrl+C, a timer): it is sent to the **process**, and the kernel picks **any one** thread that is not blocking it. "Any" is literal — never write code assuming the main thread will get it. If **every** thread blocks it, the signal stays pending at process level, interrupting nobody. That is exactly the property the solution below exploits.

Why handlers are almost unusable in a threaded program. The handler runs *inline*, on the stack of whichever thread was chosen, at an arbitrary point:

```
Thread 2:  pthread_mutex_lock(&m);   ← holds m
           x = 10;
           y = 20;   ◄── SIGALRM arrives HERE
           pthread_mutex_unlock(&m);

handler() runs on Thread 2's stack → calls lock(&m)
  → m is held by the interrupted frame of this same thread
  → the holder cannot continue until the handler returns
  → PERMANENT HANG
```

No other thread can rescue it. And it is not only your mutexes: `malloc()` and `printf()` take internal glibc locks, so calling `printf()` in a handler falls into the same trap. The bug depends on the signal landing while a lock is held — so it passes tests and dies in production.

The solution: turn asynchronous into synchronous. If "interrupting" is the source of the trouble, stop it interrupting: have a dedicated thread **fetch** signals when it is idle.

```
static sigset_t set;

static void *signal_thread(void *arg)
{
    int sig;
    for (;;) {
        sigwait(&set, &sig);           /* blocks here, costs no CPU */
        if (sig == SIGHUP)
            reload_config();           /* malloc, mutex, logging — all fine */
        else if (sig == SIGTERM || sig == SIGINT) {
            request_shutdown();
            return NULL;
        }
    }
```

```
}

int main(void)
{
    sigemptyset(&set);
    sigaddset(&set, SIGHUP);
    sigaddset(&set, SIGTERM);
    sigaddset(&set, SIGINT);
    pthread_sigmask(SIG_BLOCK, &set, NULL);   /* BEFORE any pthread_create */

    pthread_t t;
    pthread_create(&t, NULL, signal_thread, NULL);
    start_workers();                          /* they inherit the blocked mask */
}
```

The crucial point: `sigwait()` is an **ordinary function call that returns normally**, not a handler. The thread is in its own consistent state, so it may call anything.

Notes on `sigwait()`: the signal must be blocked in the calling thread, otherwise behaviour is undefined; it returns the error code directly rather than setting `errno`; and do not both `sigwait()` and install a handler for the same signal — which one gets it is unspecified.

Linux also offers `signalfd()`, turning signals into a file descriptor so they join an existing `epoll_wait()` loop — convenient when the program already has an event loop and you would rather not add a thread.

Common trap: `pthread_kill(t, SIGKILL)` does not kill only thread `t`.

```
pthread_kill(Thread 2, SIGKILL)
  → target = Thread 2
  → SIGKILL cannot be caught, ignored, or handled
  → its default action is "terminate the process"
  → ALL THREADS DIE
```

The target of a signal and the scope of its action are not the same thing. The name `pthread_kill` does not mean "kill just this thread". To end one thread, use a flag or `pthread_cancel()` (§1.12).

7.2 Threads and fork

Single-threaded, `fork()` is symmetric. Multi-threaded, it is not:

```
BEFORE fork                        AFTER fork
Parent                             Parent            Child
├── Thread 1 ◄── calls fork()      ├── Thread 1      └── Thread 1  (alone)
├── Thread 2                       ├── Thread 2
└── Thread 3                       └── Thread 3      Threads 2, 3 ✗ gone
```

The child has **only the thread that called** `fork()`. The others are not copied — they vanish with no cleanup and no destructors. But **the memory is copied intact**. That is the entire problem.

Consequence: orphaned locks.

```
Before: Thread 2 HOLDS mutex m       →  memory: m = { LOCKED, owner: Thread 2 }
After, in the CHILD: only Thread 1   →  memory: m = { LOCKED, owner: Thread 2 }
                                                                  └── does not exist
Thread 1 calls pthread_mutex_lock(&m) → waits for Thread 2 to unlock → FOREVER
```

This is not an ordinary deadlock (two parties waiting on each other). It is an **orphaned lock** — its owner evaporated.

Worse: locks you never wrote. glibc holds internal locks in `malloc()` / `free()` (arena lock), `printf()` / `fwrite()` (one per `FILE *`), `getaddrinfo()`, `syslog()`, `dlopen()`. If Thread 2 was inside `malloc()` when Thread 1 forked, the child's first `printf()` calls `malloc()`, waits for the arena lock, and hangs.

The rule POSIX states outright: between `fork()` and `exec()`, the child may call only **async-signal-safe** functions.

```
✓ read, write, close, dup2, execve, _exit
✗ malloc, printf, pthread_*
```

After `exec()`, everything is fine again. The constraint is tight enough that only two safe usages remain in practice.

7.3 Threads and I/O

The file descriptor table is shared: `fd 3` in Thread 1 is the same file as `fd 3` in Thread 2. One level deeper, **the file offset is shared too** — it lives in the open file description created by `open()`, not in any thread.

Problem 1 — racing on the file offset.

```
lseek(fd, my_offset, SEEK_SET);
read(fd, buf, len);
```

Two separate syscalls leave a gap:

```
Thread 1: lseek(fd, 0) ──────────────────── read(fd, …)  ← reads from 5000 ✗
Thread 2:              └── lseek(fd, 5000)
```

Fix with the offset-taking versions, which are atomic and do not touch the fd's offset:

```
pread(fd, buf, len, my_offset);
pwrite(fd, buf, len, my_offset);
```

Linux note: `pwrite()` ignores `O_APPEND` — do not combine the two. The alternative is for each thread to `open()` the file itself, giving each its own open file description and therefore its own offset.

Problem 2 — `errno`. It looks like a global but is not:

```
#define errno (*__errno_location())    /* glibc */
```

Each thread's `__errno_location()` returns a different address, so one thread's error cannot clobber another's. *Within* one thread it is still overwritten by the next call, so save it immediately:

```
if (write(fd, buf, n) < 0) {
    int saved = errno;      /* save NOW */
    close(tmpfd);           /* close may overwrite errno */
    return saved;
}
```

This is also why the pthreads API returns error codes instead of setting `errno` — the newer design avoids the global entirely.

Problem 3 — functions with static buffers. See §6.

---

8. Condition Variables and Barriers

8.1 The problem: busy waiting

A consumer facing an empty queue cannot pop. The bad answer is:

```
while (queue_empty()) { }     /* busy waiting / spinning */
```

The CPU is burned doing nothing but re-checking. A condition variable replaces this with a real sleep.

Tool	Answers
Mutex	"Who may enter the critical section?"
Condition variable	"Wait until when?"

They do not replace each other. They solve different problems and are always used together.

8.2 The defining property: it stores nothing

This is the most important property, and the source of nearly every related bug. A condition variable has no counter, no flag, no signal queue. It is only a rendezvous point for sleeping and waking.

Direct consequence: **if `signal()` is called while no thread is waiting, the signal is lost completely.** A thread that waits afterwards knows nothing and sleeps forever.

	Condition variable	Semaphore
Stores state	**No**	Yes (a counter)
signal/post with nobody waiting	**Lost**	Counter increments; the next wait passes straight through
External predicate required	**Mandatory**	No

Because it stores nothing, a **predicate variable is mandatory**, not optional.

8.3 The obligatory trio

```
pthread_mutex_t mtx  = PTHREAD_MUTEX_INITIALIZER;  /* 1. protects the predicate */
pthread_cond_t  cond = PTHREAD_COND_INITIALIZER;   /* 2. sleep/wake mechanism   */
int avail = 0;                                     /* 3. predicate — the state  */
```

Component	Role
Mutex	Protects the predicate from concurrent access

Component	Role
Condition variable	Sleep/wake mechanism; stores nothing
Predicate	The shared variable holding the real state — where the information actually lives

No predicate → lost wakeup, program hangs. No mutex → a race on the predicate itself.

A **predicate** is a logical expression over shared variables describing what the thread is waiting for: `avail == 0`, `count == BUF_SIZE`, `count == 0 && !done`.

8.4 Producer and consumer

```
/* Producer */                          /* Consumer */
pthread_mutex_lock(&mtx);               pthread_mutex_lock(&mtx);
data = 42;                              while (!data_ready)
data_ready = 1;                             pthread_cond_wait(&cond, &mtx);
pthread_cond_signal(&cond);             use(data);
pthread_mutex_unlock(&mtx);             pthread_mutex_unlock(&mtx);
```

8.5 What `pthread_cond_wait()` really does

It is not simply "sleep":

```
pthread_cond_wait()
  ├─► atomically release the mutex and enter the waiting state
  ├─► SLEEP
  ├─► woken by signal/broadcast
  ├─► RE-ACQUIRE the mutex
  └─► return
```

It releases the mutex while sleeping and re-acquires it before returning. That is the single most important fact about this API.

8.6 Why the mutex is required: lost wake-up

The check and the transition into waiting must happen atomically. Without the mutex:

```
Consumer                Producer
check ready = 0
                        data_ready = 1
                        signal()      ← nobody is waiting yet
sleep()                               ← sleeps forever
```

The signal arrived *before* the consumer went to sleep, and the condition variable does not remember it. With both sides using the same mutex, changing the predicate and waiting on it are coordinated, so this window cannot open.

8.7 Why `while`, never `if`

Two independent reasons.

Reason 1 — spurious wakeup. A thread may return from `pthread_cond_wait()` even though the condition is still false. With `if`, it proceeds on a false assumption; with `while`, it re-checks and waits again.

Reason 2 — multiple consumers. After a `broadcast()`, C1, C2 and C3 all wake, but only one resource exists. C1 takes it and sets `data_ready = 0`. C2 then acquires the mutex and finds the predicate false again. `while` handles this; `if` does not.

8.8 signal vs broadcast

	`signal()`	`broadcast()`
Wakes	At least one thread	Every waiting thread
Always correct?	Only under both conditions below	**Always**
Efficiency	Higher	Lower (thundering herd)

Use `signal()` only when **both** hold: only *one* thread needs to handle this change, and it does not matter *which* — every waiter does the same work.

Use `broadcast()` when waiters have different predicates, when the change can satisfy several at once, when signalling shutdown, or when you are unsure.

Thundering herd is the cost of `broadcast()`: all threads wake, the first takes the mutex and consumes the work, and the remaining N−1 each lock, find nothing to do, unlock, and sleep again — N−1 wasted context switches.

8.9 When a condition variable fits

- Shared data protected by a mutex.
- A thread must **wait** for a condition on that data, not merely get exclusive access.
- The wait is expressible as a **predicate** over shared variables.
- The moment the condition becomes true is **unpredictable**.
- The thread should really sleep (yield the CPU), not busy-wait.

---

9. Semaphores vs Mutexes

Division of labour across the three tools:

Tool	Purpose
Mutex	Protect shared state — mutual exclusion
Condition variable	Wait for shared state to reach a wanted condition
Semaphore	Count resources / permits — signalling

Two differences established elsewhere in this document:

- **Ownership.** A mutex has an owner and only the owner may unlock it. If the problem needs "thread A grants thread B permission to run" — locking in one thread and releasing in another — a mutex is the wrong tool; that is a semaphore's job (§4.3).
- **Stored state.** A semaphore has a counter, so a `post()` with nobody waiting is not lost: the next `wait()` passes straight through. A condition variable has no counter and loses it (§8.2). This is why a semaphore needs no external predicate.

---

10. Lock Implementation Concepts

Lock Implementation Concepts

The layers involved

A lock is not just a `locked` variable. Implementing one efficiently stacks several layers:

```
Application
    |
    ▼
pthread_mutex
    |
    ▼
Lock implementation
    ├── Atomic instructions
    ├── Spin / busy waiting
    └── Blocking ──► futex ──► kernel scheduler
```

Why an ordinary variable cannot implement a lock

The obvious attempt:

```
if (locked == 0) {
    locked = 1;
    /* critical section */
}
```

contains a race condition itself:

```
Thread A                Thread B

check locked == 0
                        check locked == 0
TRUE                    TRUE
locked = 1
                        locked = 1
ENTER                   ENTER
```

Both threads enter. What is needed:

> **Checking the state and changing the state must happen atomically.**

That is why atomic instructions exist.

Atomic operations

An atomic operation is observed by other threads and CPUs as an indivisible unit. The three that matter:

Compare-and-swap (CAS) — `CAS(address, expected, new)`:

```
if (*address == expected)
    *address = new;
```

with compare and conditional update performed as one operation.



Exchange — `old = *p; *p = new; return old;`. If `old == 0`, this thread just acquired the lock.

Fetch-and-add — `old = *p; *p += n; return old;`. Used for counters, reference counting, ticket locks, semaphores.

Atomicity is not the same as a lock

Atomicity of **one operation** differs from atomicity of **a sequence**:

```
if (atomic_load(&balance) >= 100) {
    atomic_fetch_sub(&balance, 100);
}
```

Still wrong — another thread can change `balance` between the check and the update. CAS is how you fuse read + check + conditional update into one operation.

Spinlock

```
atomic_int lock = 0;


void lock_acquire(void) { while (atomic_exchange(&lock, 1) == 1) { /* spin */ } }
void lock_release(void) { atomic_store(&lock, 0); }
```

The waiting thread does not sleep; it keeps re-checking, and the CPU keeps executing it the whole time.

Good when the lock is held for tens or hundreds of nanoseconds — spinning avoids the far larger cost of sleep → scheduler → context switch → wake.

Bad when the lock is held long:

```
CPU 0                CPU 1
Thread A             Thread B
holding lock         spinning
     ▼                    ▼
useful work           wasted CPU
```

Blocking lock

```
lock unavailable
   ▼
BLOCK / SLEEP        → the CPU runs another thread
   ▼
unlock → wake waiter → retry / acquire
```

The waiting thread consumes no CPU.

Spinlock vs blocking lock

Property	Spinlock	Blocking lock

Lock is busy	Spin	Sleep / block
CPU while waiting	Uses CPU	Uses none
Context switch	Not needed to wait	May occur
Very short wait	Very efficient	Overhead dominates
Long wait	Poor	More efficient
Example	`pthread_spinlock_t`	`pthread_mutex_t`

```
SPINLOCK                    BLOCKING LOCK
"The door is closed.        "The door is closed.
 I keep checking it."        I sleep; wake me when it opens."
```

Fast path and slow path

A real mutex is a hybrid. The uncontended case must never reach the kernel.

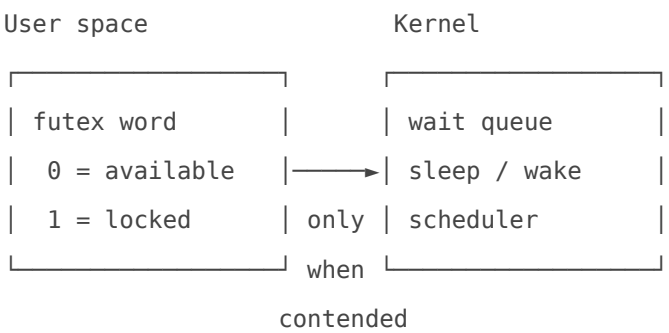
```
pthread_mutex_lock()
        ▼
   Atomic attempt
    ┌──┴──┐
SUCCESS   FAIL
   ▼       ▼
acquire   spin briefly
        ┌──┴──┐
     success  still busy
        ▼       ▼
     acquire   block ──► futex ──► kernel
```

```
Fast path  → common case → one atomic operation, no syscall
Slow path  → contention  → sleep / wakeup via the kernel
```

Futex

Fast Userspace muTEX. Not a lock type — a **building block** for implementing locks, and not the same thing as `pthread_mutex`.

The split of responsibility:



All the lock logic lives in user space on a plain word, manipulated with atomic instructions. The kernel is asked only to put a thread to sleep on an address and to wake threads sleeping on it. The uncontended case is one CAS and no system call — that is the "fast" in the name.

Futex wait and the lost wake-up

```
futex_wait(address, expected)
```

> "If the value at `address` is still `expected`, put this thread to sleep."

The `expected` argument is the whole point. Without it:

```
Thread B                Thread A


check lock = busy
                        unlock, lock = free
                        wake B      ← nobody is waiting yet
B sleeps                            ← sleeps forever
```

This is a **lost wake-up**. Because the kernel re-checks the value while holding its internal queue lock, the check and the transition into waiting become atomic, and the window closes.

This is the same reasoning as `pthread_cond_wait()` requiring a mutex.

Wake is not acquire

```
WAKE ≠ ACQUIRE
```

Between A's `futex_wake` and B actually running, thread C can take the lock. A woken thread must always retry the acquisition, and may have to block again.

Atomicity and memory ordering

Atomicity is not the whole story:

```
Atomicity       → can the operation be split or raced?
Memory ordering → in what order are reads and writes observed?
```

```
data = 42;                    while (atomic_load(&ready) == 0) ;
atomic_store(&ready, 1);      printf("%d\n", data);
```

`ready` being atomic does not make this correct. Compiler reordering, CPU reordering, and visibility all still apply.

Acquire and release

```
Release → preceding operations must be published before this operation completes
Acquire → following operations must not move ahead of this operation
```

```
Thread A                        Thread B


data = 42
release ──────────────────────► acquire
                                read data → 42
```

A lock also provides ordering



`unlock` in one thread and `lock` in another establish a synchronization relationship under the memory model. This is why a mutex is correct where `volatile` is not.

Concepts to keep distinct

Term	Meaning	Not the same as
Atomic	One operation performed indivisibly	A lock
Spinlock	Lock busy → keep checking, burn CPU	—
Blocking lock	Lock busy → sleep, yield the CPU	—
Futex	Kernel wait/wake mechanism for building primitives	`pthread_mutex`
Mutex	High-level primitive giving mutual exclusion **and** ordering	Any one implementation

A mutex underneath is: atomic operations + brief spinning + futex blocking.

Mental model

```
Atomic instruction   → "change state safely"
       ↓
Spinlock             → "can't get it → keep trying"
       ↓
Blocking lock        → "can't get it → go to sleep"
       ↓
Futex                → "the kernel handles sleep/wake"
       ↓
Mutex                → the primitive the application uses
```

Further topics

Memory model: C11 atomics, the five `memory_order_*` values, compiler and CPU reordering, memory barriers, happens-before.

Lock implementation: adaptive spinning, spurious wake-up, fairness and starvation, priority inversion and priority inheritance.

Advanced locks: ticket lock, MCS lock, seqlock, RCU.

Lock-free: CAS loops, wait-free vs lock-free, the ABA problem, lock-free queues and stacks.

Hardware: x86 vs ARM atomic instructions, cache coherence and MESI, false sharing, CPU memory models, Linux futex internals.

Of these, **memory ordering → acquire/release → happens-before** is the one worth a chapter of its own: it is the link between C code, the compiler, and the CPU, and it explains why code that looks correct can still fail on a multi-core system.

Quick Reference

Question	Answer
What actually creates a thread?	`clone()` with `CLONE_VM|CLONE_FILES|CLONE_THREAD|…`
Why is thread creation ~10× faster than `fork()`?	`CLONE_VM` — no new page table is built
Does the parent run before the child?	No guarantee, ever — use `pthread_self()`
How should a thread get its own ID?	`pthread_self()`, never the variable `pthread_create()` wrote
Why pass `arg` if memory is shared?	Removes the race, makes the routine reusable, avoids locks
Why does `detach` exist?	Join blocks, and some designs have no thread free to wait
Why is `volatile` not enough?	It gives no atomicity, no happens-before, no mutual exclusion
What does a mutex give beyond exclusion?	Atomicity of a sequence, and a release/acquire memory barrier
Why must `pthread_cond_wait()` take a mutex?	To make "check the predicate" and "start waiting" atomic — otherwise lost wake-up
Why `while`, not `if`, around `cond_wait`?	Spurious wakeups, and multiple consumers competing for one resource
Why is `strtok()` unsafe with threads?	Its saved position is `static` — shared even when the strings are not
What can the child call after `fork()` in a threaded program?	Only async-signal-safe functions, until `exec()`
Does `pthread_kill(t, SIGKILL)` kill only `t`?	No — SIGKILL's default action terminates the whole process

```
ulimit -s                        # default thread stack size (RLIMIT_STACK)
ulimit -u                        # RLIMIT_NPROC
cat /proc/sys/kernel/threads-max
cat /proc/sys/vm/max_map_count   # ceiling on stack mappings
ps -eLf                          # tasks including threads (LWP column)
grep Threads /proc/PID/status

gcc -pthread -fsanitize=thread -g app.c
valgrind --tool=helgrind ./app
```

LAB

LAB1

https://github.com/trungnm2111/session4_race_condition

LAB2, 3

https://github.com/trungnm2111/session4_reentrancy

LAB4

https://github.com/trungnm2111/session4_producer-consumer

LAB5

https://github.com/trungnm2111/session4_deadlock

Revision #6
Created 2026-08-28 01:56:29 UTC by trungnguyen
Updated 2026-09-07 02:15:52 UTC by trungnguyen